Evaluating Checkpoint/Restore-Based Pod Migration in Kubernetes: An Empirical Performance and Bottleneck Study

Musa Khasmammadov
Independent Systems Researcher
Baku, Azerbaijan
musaxasmammedov77@gmail.com

Abstract—Stateful singleton containers lacking distributed consensus lose volatile heap state during Kubernetes node maintenance and cloud Spot instance preemptions. We evaluate an automated, snapshot-consistent pod migration pipeline coordinating CRIU (v4.3), the Kubelet Checkpoint API (KEP-2008), Buildah OCI packaging, CRI-O (v1.31), and Cilium eBPF routing across AWS EC2 nodes. Across 36 decoupled migrations compared against 20 native cold reschedules of a stateful service, median cutover blackout is 1064.5 ms versus 1019.0 ms (+4.5%), while sample means show an overhead of 125.4 ms (+12.5%, 1132.4±241.1 ms vs. 1007.0±231.2 ms, 95% CI: $[-6.7, +257.4]$ ms; Hodges-Lehmann difference: 136.0 ms; Mann-Whitney $U = 252.5, p = 0.0660$). Paired concurrent probing shows external recovery via Service follows operator reporting with a mean lag of +15.8 ms (95% CI $[+11.0, +20.6]$ ms) for CRIU and +18.5 ms ($[+12.5, +24.5]$ ms) for cold (per-run range over all 56 runs: −12 to +48 ms), consistent with sampling jitter of 50 ms polling and 20 ms probing. Blackout variance is governed by Kubelet status reporting (1 s PLEG relisting and status updates) rather than container reinstantiation, with CRI-O unfreezing at 478–516 ms. In exploratory scaling up to 1024 MB (1 GB), uncompressed Buildah packaging slashes build time from 32.06 s to 13.74 s (2.3×), completing source evacuation in 36.78 s within the 120 s Spot deadline with 100% SHA-256 bit integrity, while writeback throughput (137.9 MiB/s) is consistent with EBS gp3 baseline storage limits (125 MiB/s). Finally, single-container KEP-2008 checkpointing severs in-pod loopback sockets, favoring node-level eBPF routing.

Index Terms—Kubernetes, Container Checkpoint/Restore, CRIU, CRI-O, Cilium eBPF, Stateful Workloads, Spot Instances, Recovery Point Objective.

I. Introduction

Kubernetes manages workloads under the foundational assumption that container instances are ephemeral and stateless [1]. When worker nodes undergo kernel maintenance (kubectl drain) or cloud virtual machines are reclaimed, stateless microservices maintain availability by spinning up replacement replicas before terminating old instances (maxSurge).

However, critical production classes maintain large, mutable in-memory heaps without built-in distributed consensus:

1) Scientific & Numerical Solvers: Iterative physics simulations, linear algebra solvers, and legacy C++/Python runtimes accumulating state over hours.
2) Warm ML Inference Caches & Aggregators: High-dimensional embedding buffers, key-value transformer caches, and session aggregators where rebuilding state from cold storage imposes severe warm-up latency.

Deploying these workloads on Amazon Web Services (AWS) EC2 Spot instances yields 70–90% compute cost savings [2], [17], but subjects pods to abrupt reclamation with a 120-second termination notice [3]. Standard eviction issues SIGTERM followed by SIGKILL, destroying volatile RAM. Offloading state to external tiers such as Redis [4] prevents data loss, but incurs intrusive architectural refactoring and replaces nanosecond memory dereferences with network roundtrips.

Userspace checkpoint/restore via CRIU [5] provides an application-transparent alternative. In hypervisor literature, "live migration" classically refers to iterative pre-copy schemes [10] that track dirty pages via hardware MMU page tables, streaming memory while the virtual machine executes to achieve millisecond cutovers. In container environments lacking hardware virtualization traps, process-level live migration via CRIU and Kubernetes KEP-2008 [8] is operationally distinct. In this paper, we use the term checkpoint/restore-based container migration to denote cross-node state transfer that preserves running process heaps and listening sockets without re-executing main() or rebuilding application memory structures, while accepting a sub-second cutover blackout bounded by snapshot serialization and container runtime event loops.

This paper makes four empirical systems contributions:

- End-to-End Controller: Privileged DaemonSet operator (PodLiveMigration CRD) automating KEP-2008 checkpointing, Buildah packaging, registry transfer, CRI-O restore, and Cilium eBPF routing across AWS EC2.
- Statistical Evaluation & Runtime Profiling ($N = 36$ CRIU vs. $N = 20$ Cold): Under paired 50 Hz probing, inferential tests ($p = 0.0660$, Hodges-Lehmann: 136.0 ms; Welch's $t = 1.91, p = 0.0628$) and tick

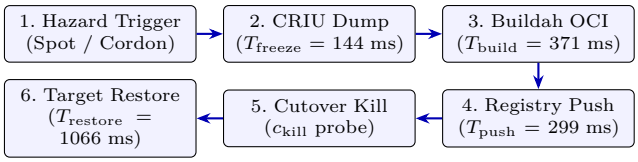


Fig. 1. Decoupled snapshot-consistent migration pipeline across nodes.

regression ($r = 0.9624$) show blackout variance is dominated by Kubelet status reporting. Direct daemon profiling confirms container unfreezing at 478–516 ms; paired concurrent probing shows external recovery via Service follows operator reporting with a mean lag of +15.8 ms (95% CI $[+11.0, +20.6]$ ms) for CRIU and +18.5 ms ($[+12.5, +24.5]$ ms) for cold (per-run range over all 56 runs: −12 to +48 ms), consistent with sampling jitter of 50 ms polling and 20 ms probing.

- 1 GB Scaling & Packaging Optimization: In exploratory scaling up to 1024 MB ($N = 1$), 100% SHA-256 bit integrity is verified. We observe writeback throughput (137.9 MiB/s) consistent with EBS gp3 baseline storage limits (125 MiB/s [18]) at 1 GB, and show uncompressed Buildah packaging slashes build time from 32.06 s to 13.74 s (2.3×, completing evacuation in 36.78 s within the 120 s Spot deadline).
- State Reconstruction Benefit & Sidecars: We contrast checkpointing against analytical cold state reconstruction (11–30 s for 1 GB), analyze RPO trade-offs, and profile single-container KEP-2008 sidecar loopback constraints.

II. System Architecture

A. Pipeline Design and CRIU Configuration

Our testbed runs on Ubuntu 24.04 LTS (Linux 6.8) with CRIU v4.3 [5], Kubelet v1.31.14 (KEP-2008 enabled) [8], CRI-O v1.31.4 with the runc runtime, Buildah, and Cilium v1.16.3 (kubeProxyReplacement=true) [9]. Figure 1 details the six-stage migration pipeline.

Because a restored pod receives a new Pod IP from the target node's CIDR (10.244.2.0/24 → 10.244.1.0/24), /etc/criu/default.conf specifies key flags [5]:

- tcp-close: Restores listening sockets (0.0.0.0:8090) in the LISTEN state while closing established TCP sockets bound to the old Pod IP.
- skip-in-flight: Drops transient in-flight TCP handshakes occurring during 50 Hz probing at the instant of ptrace seizure.
- skip-mnt /run/.containerenv: Skips CRI-O's transient metadata mount so CRImportCheckpoint mount validation succeeds.
- ext-unix-sk and shell-job: Permits dumping external UNIX domain sockets and inherited stdio descriptors.

B. Cutover Blackout Definition

Our benchmark service uses HTTP/1.0 behind a Kubernetes Service ClusterIP and Cilium L7 Envoy Ingress without long-hold request buffering. Between source pod deletion (t_{kill}) and target endpoint readiness (t_4), incoming requests fail immediately with connection refused or HTTP 503. We define cutover blackout T_{blackout} strictly as the unavailability window during which probes fail, measured both by the operator clock ($t_4 - t_{\text{kill}}$) and by an external 50 Hz (±20 ms) prober as the maximum interval between consecutive HTTP 200 OK responses.

III. Latency, State Drift, and Evacuation Model

A. Stage Timestamps and Disruption Decomposition

The operator records monotonic timestamps around each pipeline step: t_0 (pre-checkpoint), t_1 (post-dump), t_2 (post-build), t_3 (post-push), t_{kill} (pre-delete probe c_{kill} and pod deletion), and t_4 (first HTTP 200 OK from the restored pod):

$$T_{\text{freeze}} = t_1 - t_0, \quad T_{\text{build}} = t_2 - t_1, \quad T_{\text{push}} = t_3 - t_2 \tag{1}$$

$$T_{\text{restore}} = t_4 - t_3, \quad T_{\text{blackout}} = t_4 - t_{\text{kill}} = T_{\text{restore}} - t_{\text{probe}} \tag{2}$$

where $t_{\text{probe}} = t_{\text{kill}} - t_3 \approx 1$–2 ms is pre-kill probe latency. Client-perceived disruption consists of source freeze stutter T_{freeze} (threads paused by ptrace) plus cutover blackout T_{blackout}:

$$T_{\text{disruption}} = T_{\text{freeze}} + T_{\text{blackout}} \tag{3}$$

$$T_{\text{total}} = T_{\text{freeze}} + T_{\text{build}} + T_{\text{push}} + T_{\text{restore}} \tag{4}$$

B. State Drift (RPO), Post-Restore Ticks, and Rollback

In Mode A (Decoupled, KeepRunning: true), CRI-O unfreezes the source container at t_1, allowing it to serve traffic while Buildah packages and pushes the image. State mutations occurring between the snapshot ($S_{\text{ckpt}} \approx c_{\text{dump}} + 1$) and pod deletion ($t_{\text{kill}}$) are lost upon cutover. At mutation rate λ, Recovery Point Objective (RPO) and lost updates ΔS_{lost} are:

$$\text{RPO} = T_{\text{build}} + T_{\text{push}} \tag{5}$$

$$\Delta S_{\text{lost}} = c_{\text{kill}} - S_{\text{ckpt}} \approx \lambda \cdot (T_{\text{build}} + T_{\text{push}}) \tag{6}$$

Workloads with external side effects require idempotency keys or write-ahead logging (WAL).

In Mode B (Synchronous Stop-and-Copy, Analytical Projection), halting the source container at t_1 yields RPO ≡ 0, but extends service unavailability to the entire pipeline ($T_{\text{blackout}}^{\text{stop-and-copy}} = T_{\text{total}}$).

On the target node, runc restore unfreezes the process at $T_{\text{unfreeze}} < T_{\text{restore}}$, and the restored process immediately resumes incrementing from S_{ckpt}. However, the operator only probes c_{restore} after Kubelet's PLEG loop (1 s period) updates Pod.status.phase = Running at $T_{\text{restore}} = T_{\text{unfreeze}} + \Delta t_{\text{pleg}}$:

$$c_{\text{restore}} - c_{\text{dump}} \approx \lambda \cdot (T_{\text{restore}} - T_{\text{unfreeze}}) \tag{7}$$

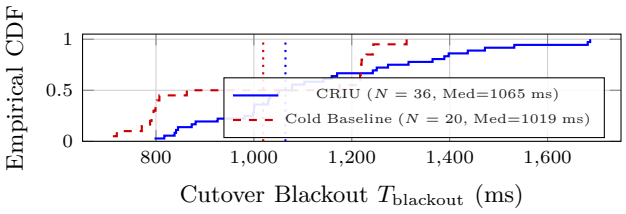


Fig. 2. Empirical CDF of Cutover Blackout (T_{blackout}). Dotted lines mark sample medians (CRIU: 1064.5 ms, Cold Baseline: 1019.0 ms).

C. Source-Side Spot Evacuation Condition

When AWS issues a 120-second Spot Interruption Notice, only the source node is terminated at $t = 120$ s; target-side pull and restoration execute on the surviving destination node after t_3. Spot survival requires only that worst-case source evacuation $T_{\text{evac}}^{\text{worst}}(M)$ complete within 120 s:

$$T_{\text{evac}}^{\text{worst}}(M) = T_{\text{detect}}^{\text{worst}} + T_{\text{freeze}}(M) + T_{\text{build}}(M) + T_{\text{push}}(M) < 120 \text{ s} \quad (8)$$

where $T_{\text{detect}}^{\text{worst}} = 5.0$ s accounts for the worst-case detection latency under a 5 s IMDSv2 polling period.

IV. Empirical Evaluation

A. Testbed Setup

Experiments were conducted on a three-node AWS EC2 cluster in eu-central-1: one c7i-flex.large control plane (2 vCPU, 4 GB RAM, hosting the OCI registry at 10.10.1.10:5000 and Cilium Ingress) and two m7i-flex.large workers (2 vCPU, 8 GB RAM, EBS gp3 storage). All 11 microservices of Google Online Boutique ran continuously to generate background multi-tenant CPU, memory, and CNI contention.

B. Experiment 1: Statistical Evaluation ($N = 36$ CRIU vs. $N = 20$ Cold Baseline)

Table I summarizes measurements across $N = 36$ CRIU migrations ($n = 20$ w1 $\rightarrow$ w2; $n = 16$ w2 $\rightarrow$ w1) and $N = 20$ native cold reschedules of the 26 MB stateful service ($\lambda = 10$ Hz). Table II presents individual trace breakdowns and the Mode B projection. Figure 2 displays the empirical cumulative distribution function (ECDF) of cutover blackout.

Three crucial findings emerge:

1) Cutover Blackout and Paired Dual-Prober Validation: Median operator blackout is 1064.5 ms for CRIU vs. 1019.0 ms for cold restart (+4.5%), while means show an overhead of 125.4 ms (+12.5%, 1132.4 ± 241.1 ms vs. 1007.0 ± 231.2 ms, 95% CI $[-6.7, +257.4]$ ms; Hodges-Lehmann diff: 136.0 ms; $U = 252.5, p = 0.0660$; Welch's $t(40.8) = 1.91, p = 0.0628$; CIs of cohort means are normal-approximations, whereas the difference CI is Welch t-based). The operator polls the Kubernetes apiserver for Pod.status.phase = Running at 50 ms intervals before probing Pod IP directly, while an external 50 Hz prober concurrently queries ClusterIP. Across 56 paired runs, external probing lags operator reporting by a mean of +15.8 ms for CRIU (95% CI $[+11.0, +20.6]$ ms) and +18.5 ms for cold restart (95% CI $[+12.5, +24.5]$ ms; external means diff: 122.7 ms, $U = 252.0, p = 0.0648$). The paired differences ($\sigma = 14.7$ ms for CRIU, $\sigma = 13.8$ ms for cold; per-run range over all 56 runs: -12 to +48 ms) are consistent with the theoretical sampling convolution of the 50 ms apiserver polling loop ($50/\sqrt{12} \approx 14.4$ ms) and the 20 ms prober grid ($20/\sqrt{12} \approx 5.8$ ms; theoretical joint $\sigma \approx \sqrt{14.4^2 + 5.8^2} \approx 15.5$ ms). Under this sampling model, the observed mean lag reflects tens of milliseconds of Service routing propagation. Because the manifest omits a custom readinessProbe, Kubernetes sets PodReady = True in the same status update as Running (which endpointslice-controller observes via watch without intermediate probe polling).
2) PLEG Dominance and Container Runtime Datapath: Standard deviations for CRIU ($\sigma = 241.1$ ms) and cold restart ($\sigma = 231.2$ ms) are consistent in order of magnitude with uniform 1 s PLEG relist jitter ($1000/\sqrt{12} \approx 288.7$ ms). Linear tick regression yields $r = 0.9624$ and x-intercept $T_{\text{unfreeze}} = 537.1$ ms, which direct CRI-O daemon profiling confirms at 478–516 ms (journalctl -u crio: StartContainer: $\approx$ 310 ms pull, $\approx$ 168 ms restore). Rather than routing convergence, the residual 261–1151 ms delay after unfreeze is consistent with Kubelet status reporting (interplay of PLEG, status manager, and apiserver synchronization).
3) Trace Dynamics and Methodological Threats: In Trace #35 (51 $\rightarrow$ 60 $\rightarrow$ 57), 5 post-restore ticks accumulated during 543 ms of PLEG delay, whereas Trace #36 (77 $\rightarrow$ 85 $\rightarrow$ 79) experienced only 1 tick. Runs were executed in directional series ($n = 20$ w1$\rightarrow$w2, $n = 16$ w2$\rightarrow$w1) with low dirtying rates; non-randomized execution and write-heavy workloads represent threats to validity.

C. Experiment 2: Empirical Heap Scaling, SHA-256 Verification, and Gzip Optimization

We populated a contiguous heap buffer of $M \in \{64, 128, 256, 512, 1024\}$ MB with high-entropy pseudo-random float64 weights (simulating an ML embedding table or scientific grid). Prior to checkpointing on k8s-worker-1, the service computed the SHA-256 digest over the entire heap; upon restoration on k8s-worker-2, the /live/verify endpoint recomputed and validated the digest. Table III and Figure 3 present the measurements.

Three systems insights emerge:

1) Bit-Exact Restoration within Spot SLA: In single-trial scaling ($N = 1$), all heaps up to 1024 MB restored with 100% SHA-256 integrity. Under worst-case Spot

TABLE I
Statistical Evaluation: $N = 36$ CRIU Decoupled Pod Migrations vs. $N = 20$ Native Cold Reschedules (26 MB Stateful Service)

Configuration / Subset	T_{freeze} (ms)	T_{build} (ms)	T_{push} (ms)	RPO (ms)	T_{restore} (ms)	T_{blackout} (ms)	$T_{\text{disruption}}$ (ms)	T_{total} (ms)	$c_{\text{kill}} - c_{\text{dump}}$ (ticks)
CRIU w1 → w2 ($n = 20$)									
Median ($p50$) / $p95$	144.0 / 158	368.5 / 400	293.0 / 310	664.0 / 703	1033.0 / 1533	1031.5 / 1532	1178.0 / 1673	1857.0 / 2358	7.5 / 8
Mean ± SD [Min–Max]	143.3 ± 9.4	370.9 ± 17.1	293.6 ± 12.5	664.6 ± 27.6	1112.9 ± 249.2	1111.2 ± 249.3 [817–1687]	1254.5 ± 250.1 [958–1836]	1920.8 ± 248.7	7.6 ± 0.6 [7–9]
CRIU w2 → w1 ($n = 16$)									
Median ($p50$) / $p95$	145.0 / 155	372.5 / 384	301.5 / 308	672.5 / 688	1092.0 / 1474	1090.5 / 1472	1229.0 / 1622	1887.0 / 2277	7.0 / 8
Mean ± SD [Min–Max]	145.4 ± 7.1	366.6 ± 16.4	296.6 ± 12.4	663.2 ± 27.9	1160.2 ± 235.9	1158.8 ± 235.7 [797–1682]	1304.1 ± 238.1 [944–1828]	1968.8 ± 238.0	7.4 ± 0.5 [7–8]
CRIU Combined ($N = 36$)									
Median ($p50$) / $p95$	144.0 / 156	370.5 / 389	298.5 / 310	669.0 / 696	1066.0 / 1533	1064.5 / 1532	1203.0 / 1673	1878.0 / 2358	7.0 / 8 (RAM Kept)
Mean ± SD [95% CI]	144.2 ± 8.4	369.0 ± 16.7	294.9 ± 12.4	663.9 ± 27.4	1133.9 ± 241.1	1132.4 ± 241.1 [1053.6, 1211.1]	1276.6 ± 242.6	1942.1 ± 241.8	7.5 ± 0.6 [7–9]
Native Cold Baseline ($N = 20$)									
Median ($p50$) / $p95$	N/A	N/A	N/A	Total Loss	1019.0 / 1244	1019.0 / 1244	1019.0 / 1244	1019.0 / 1244	100% State Lost
Mean ± SD [95% CI]	N/A	N/A	N/A	Total Loss	1007.0 ± 231.2	1007.0 ± 231.2 [905.7, 1108.3]	1007.0 ± 231.2	1007.0 ± 231.2	Reset to 0 [1–6]

Inferential Tests (T_{blackout}): Mann-Whitney $U = 252.5, p = 0.0660$; Hodges-Lehmann median diff = +136.0 ms (95% CI [−18.0, +252.0] ms); Welch's $t(40.8) = 1.91, p = 0.0628$; Mean diff = +125.4 ms (95% CI [−6.7, +257.4] ms, Welch t). CIs of cohort means are normal-approximations. External 50 Hz Prober ($T^{\text{ext}}_{\text{blackout}}$): CRIU median 1080.0 ms (1148.2 ± 241.6 ms) vs. Cold 1040.0 ms (1025.5 ± 231.3 ms).

TABLE II
Detailed Trace Breakdown and Operating Mode Trade-offs (RTO vs. RPO)

Mode / Trace ID	Path	T_{total}	T_{freeze}	T_{build}	T_{push}	T_{blackout}	$T_{\text{disruption}}$	RPO	Counter($c_{\text{dump}} \to c_{\text{kill}} \to c_{\text{restore}}$)
Cold Reschedule (UI Trace)	w2 → w1	1289 ms	N/A	N/A	N/A	1289 ms	1289 ms	Total Loss	43544 → 8 (Lost 43536 ticks)
CRIU Decoupled (Trace #35)	w1 → w2	1935 ms	159 ms	385 ms	310 ms	1078 ms	1237 ms	695 ms	51 → 60 → 57 ($\Delta S_{\text{lost}} \approx 8, \Delta S_{\text{post}} \approx 5$)
CRIU Decoupled (Trace #36)	w2 → w1	1642 ms	147 ms	388 ms	308 ms	797 ms	944 ms	696 ms	77 → 85 → 79 ($\Delta S_{\text{lost}} \approx 7, \Delta S_{\text{post}} \approx 1$)
Mode B Stop-and-Copy (Model Projection)	Bidirectional	1878 ms	144 ms	370.5 ms	298.5 ms	1878 ms	1878 ms	0 ms	Zero Drift ($T_{\text{blackout}} = T_{\text{total}}$)

TABLE III
Empirical Memory Scaling on AWS EC2 (26 MB to 1024 MB Heap) with Cross-Node SHA-256 Heap Integrity Verification

Allocated Heap	OCI Layer Size	T_{freeze} (Dump)	T_{build} (gzip)	T_{build} (-D)	RPO (gzip / -D)	T_{evac} (5 s budget)	% 120s SLA	T_{restore}	T_{total} (gzip / -D)	SHA-256 Digest
26 MB ($N = 36$ Med.)	5.16 MiB	0.144 s	0.371 s	0.261 s	0.669 s / 0.560 s	5.81 s / 5.70 s	4.8% / 4.8%	1.066 s	1.88 s / 1.77 s	Verified (Counter)
64 MB (Weights)	69.11 MiB	0.216 s	0.761 s	0.361 s	1.376 s / 0.976 s	6.59 s / 6.19 s	5.5% / 5.2%	0.951 s	2.54 s / 2.14 s	100% Match (7785bfc7)
128 MB (Weights)	133.11 MiB	0.258 s	1.082 s	0.528 s	2.011 s / 1.457 s	7.27 s / 6.71 s	6.1% / 5.6%	1.695 s	3.96 s / 3.41 s	100% Match (cce52f48)
256 MB (Weights)	261.13 MiB	0.375 s	5.796 s	1.473 s (3.9×)	7.481 s / 3.158 s	12.86 s / 8.53 s	10.7% / 7.1%	2.108 s	9.96 s / 5.64 s	100% Match (4b1b1359)
512 MB (Weights)	517.15 MiB	0.760 s	16.120 s	4.210 s (3.8×)	19.387 s / 7.477 s	25.15 s / 13.24 s	21.0% / 11.0%	7.807 s	27.95 s / 16.04 s	100% Match (ab028a06)
1024 MB (1 GB)	1029.21 MiB	7.464 s	32.060 s	13.743 s (2.3×)	42.630 s / 24.313 s	55.09 s / 36.78 s	45.9% / 30.6%	17.328 s	67.42 s / 49.11 s	100% Match (c0da7edf)

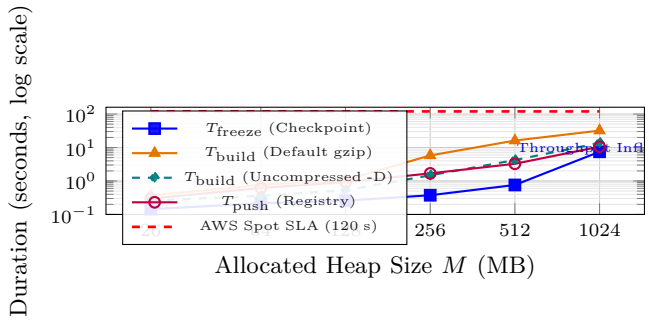


Fig. 3. Empirical heap scaling on AWS EC2 (26–1024 MB), consistent with EBS gp3 baseline storage limits and showing Buildah uncompressed packaging optimization ($N = 1$).

detection ($T^{\text{worst}}_{\text{detect}} = 5.0$ s) and gzip, source evacuation took 55.09 s (45.9% of the 120 s SLA). Between 512 MB and 1024 MB, T_{freeze} jumped 9.8× (coincident with dirty page accumulation) and T_{push} grew 3.2×, while gzip T_{build} scaled roughly proportionally between 512 MB and 1 GB (2.0×, whereas smaller heaps scaled more steeply, e.g., 5.4× from 128 to 256 MB). Projecting to 2 GB suggests default gzip nears the 120 s deadline (≈ 105–130 s), requiring uncompressed packaging for multi-gigabyte state.

2) EBS gp3 Storage Throughput and Writeback in T_{freeze}: From 26 MB to 512 MB, T_{freeze} remains sub-second (144 → 760 ms) via page cache buffering. At 1 GB, uncommitted dirty memory hits 783.6 MB (vm.dirty_background_ratio = 10), triggering kernel flushers. The observed write throughput of 1029.2 MiB/7.464 s ≈ 137.9 MiB/s (144.6 MB/s) is consistent with AWS EBS gp3 baseline limits (125 MiB/s [18]) combined with page caching and co-located service memory pressure.

3) Gzip Incompressibility and –disable-compression Speedup: Incompressible weights produce OCI layers matching heap size. Single-threaded gzip T_{build} takes 32.060 s at 1 GB. Passing –disable-compression (-D) slashes packaging to 13.743 s (2.3× speedup), trimming evacuation to 36.78 s (30.6% of Spot budget). Over VPC links (1029 MiB pulls in 8.5 s over 1 Gbps baseline), disabling compression is optimal.

D. Why Migrate? Checkpoint Restoration vs. Cold State Rebuild

A cold reschedule launches an empty container in ≈ 1.0 s but resets memory state to zero. As an analytical baseline, downloading a 1 GB snapshot from Amazon S3 over 300–800 Mbps takes 10.2–27.3 s transfer plus 1.05–2.5 s deserialization (11.2–29.8 s total recovery). At 1 GB, migration cutover ($T_{\text{restore}} = 17.328$ s) is comparable to cold reconstruction while bounding state drift (24.3 s under -D vs. 42.6 s under gzip; stop-and-copy: 49.1 s vs. 67.4 s). For heaps ≤ 256 MB, cutover is 0.95–2.11 s (drift: 0.56–3.16 s under -D vs. 0.67–7.48 s under gzip; stop-and-copy: 1.77–5.64 s vs. 1.88–9.96 s). Target restore breaks down into: (1) VPC pull (≈ 8.5 s), (2) CRI-O extraction (≈ 4.8 s), (3) runc/criu restore unfreezing (≈ 3.2 s), and (4) Kubelet status reporting (≈ 0.8 s); layer transfer and unpacking govern > 76% of restore latency.

E. Architectural Limitations of In-Pod Sidecars

In a two-container pod with stream-server (:8090) and an Envoy sidecar (:15006) over loopback TCP, KEP-2008

dumped the container in 118 ms and CRI-O restored it on k8s-worker-2. However, because KEP-2008 operates per container and CRIU closes established TCP sockets (tcp-close), the restored application was isolated from its proxy ($T_{\text{blackout}} = \infty$, timing out at 120 s). Node-level routing (Cilium eBPF host routing) decouples proxy lifecycles, avoiding sidecar fate-sharing.

V. Related Work

Clark et al. [10] established VM pre-copy live migration. Voyager [7] and Stoyanov and Kollingbaum [6] combined CRIU with layered storage. Reber [8] standardized KEP-2008 in Kubernetes. Ma et al. [15] analyzed layered storage, and Puliafito et al. [16] evaluated fog migration. Recent works address edge and cloud migration: Zhang et al. [11] (KubeSPT), Calagna et al. [12] (edge migration), Bellavista et al. [13] (ML workloads), and Poggiani et al. [14] (multi-cluster Liqo/CRIU). Complementing these, our study provides an empirical AWS EC2 bottleneck analysis quantifying upstream KEP-2008 and CRI-O bottlenecks: Kubelet status reporting jitter, EBS storage limits, and uncompressed OCI packaging scaling up to 1 GB.

VI. Limitations, Threats to Validity, and Security

Threats to Validity: We acknowledge key limitations: (1) Workload realism: monotonic counters exhibit minimal dirtying; write-heavy workloads increase dirtying rates ($\dot{M}_{\text{dirty}}$) and drift. Persistent connections require ingress buffering or –tcp-established. Multi-gigabyte heaps benefit from criu page-server streaming. (2) Sample size: heap scaling is exploratory ($N = 1$); isolating writeback dynamics requires multi-trial tmpfs and provisioned gp3 controls. (3) Residual delay decomposition: Service recovery follows operator reporting with mean lag +15.8 ms (CRIU) and +18.5 ms (cold), reflecting tens of milliseconds of Service propagation under 50/20 ms sampling jitter; isolating Kubelet status reporting requires probing Pod IP from t_3 rather than awaiting Running. (4) S3 recovery and Mode B are analytical projections. (5) Statistical power: with $N = 36$ and $N = 20$ ($\sigma \approx 240$ ms), 80% power ($\alpha = 0.05$) detects only differences above $\approx$ 190 ms; the observed 125 ms mean difference is inconclusive rather than proof of equivalence. (6) Spot scenario: 120 s headroom is a budget calculation from measured phases and assumed 5 s notice; no real Spot interruption was exercised. Security Considerations: Checkpoint memory dumps contain plaintext application secrets. In our prototype, checkpoint images are pushed to an unauthenticated cluster registry, and the operator runs as a privileged DaemonSet utilizing nsenter. Production deployment necessitates: (1) cryptographic checkpoint encryption at rest, (2) mutual TLS and signed OCI images, and (3) replacing root DaemonSets with scoped capabilities (CAP_CHECKPOINT_RESTORE [19], CAP_SYS_PTRACE).

VII. Conclusion

We presented an empirical study of checkpoint/restore pod migration in Kubernetes using CRI-O, CRIU v4.3, Buildah, and Cilium eBPF. Across $N = 36$ migrations and $N = 20$ cold reschedules on AWS EC2, cutover blackout showed an overhead of 45.5 ms in median (+4.5%) and 125.4 ms in mean (+12.5%, 95% CI $[-6.7, +257.4]$ ms; Hodges-Lehmann diff: 136.0 ms; $p = 0.0660$). Variance is governed by Kubelet status reporting (1 s PLEG relisting and status manager synchronization) rather than container memory instantiation, with CRI-O profiling confirming unfreezing at 478–516 ms, while external recovery via Service follows operator reporting with a mean lag of +15.8 ms (CRIU) and +18.5 ms (cold; per-run range over all 56 runs: −12 to +48 ms), consistent with tens of milliseconds of Service propagation and joint 50 ms/20 ms sampling jitter. In exploratory scaling up to 1024 MB, uncompressed packaging slashes build time from 32.06 s to 13.74 s (2.3×, evacuating in 36.78 s within the 120 s Spot deadline) with 100% SHA-256 integrity, while writeback throughput (137.9 MiB/s) aligns with EBS gp3 baseline storage limits at 1 GB. Checkpointing eliminates external persistence tiers, trading bounded state drift (0.56–3.16 s under -D vs. 0.67–7.48 s under gzip) for 1.77–5.64 s stop-and-copy cutover at $\leq$ 256 MB under -D (1.88–9.96 s under gzip). Single-container checkpointing severs loopback sidecars, favoring node-level routing.

Artifact Availability

Manifests, operator code, raw CSV data for all 56 paired runs, and reproduction statistical scripts are open-source (release v1.3.0): https://github.com/musaxasmammedov7/final-research.

References

[1] B. Burns et al., "Borg, Omega, and Kubernetes," CACM, vol. 59, no. 5, pp. 50–57, 2016.
[2] Netflix TechBlog, "Creating Your Own EC2 Spot Market," Netflix Inc., 2015.
[3] AWS, "Spot Instance Interruptions and EC2 Rebalance Recommendations," 2024.
[4] Redis Ltd., "Redis Latency Problems Troubleshooting," https://redis.io, 2024.
[5] CRIU Authors, "CRIU CLI Options and Architecture," https://criu.org, 2024.
[6] R. Stoyanov and M. Kollingbaum, "Efficient Live Migration of Linux Containers," in Proc. ISC Workshops, Springer, 2018, pp. 184–193.
[7] S. Nadgowda et al., "Voyager: Complete Container State Migration," in Proc. IEEE ICDCS, 2017, pp. 2137–2142.
[8] A. Reber, "Forensic Container Checkpointing in Kubernetes," Kubernetes Blog, Dec. 2022.
[9] Cilium Authors, "Cilium: eBPF Networking and Service Mesh," Linux Foundation, 2024.
[10] C. Clark et al., "Live Migration of Virtual Machines," in Proc. USENIX NSDI, 2005, pp. 273–286.
[11] H. Zhang et al., "KubeSPT: Stateful Pod Teleportation for Resilience With Live Migration," IEEE Trans. Serv. Comput., vol. 18, no. 3, pp. 1500–1514, 2025.
[12] A. Calagna et al., "Design, Modeling, and Implementation of Robust Migration of Stateful Edge Microservices," IEEE TNSM, vol. 21, no. 2, pp. 1877–1893, 2024.
[13] P. Bellavista et al., "Kubernetes Enhanced Stateful Service Migration for ML Applications," in Proc. IEEE AIoT, 2024.
[14] L. Poggiani et al., "Live Migration of Multi-Container Pods in Serverless Edge Systems," in Proc. ACM SEATED, 2024.
[15] L. Ma, S. Yi, N. Carter, and Q. Li, "Efficient Live Migration of Edge Services Leveraging Container Layered Storage," IEEE Trans. Mob. Comput., vol. 18, no. 9, pp. 2020–2033, 2019. DOI: 10.1109/TMC.2018.2871842.
[16] C. Puliafito et al., "Container Migration in the Fog: A Performance Evaluation," Sensors, vol. 19, no. 7, p. 1488, 2019. DOI: 10.3390/s19071488.
[17] P. Sharma, T. Guo, X. He, D. Irwin, and P. Shenoy, "Flint: Batch-Interactive Data-Intensive Processing on Transient Servers," in Proc. 11th ACM EuroSys, 2016.
[18] AWS, "Amazon EBS Volume Types: General Purpose SSD Volumes (gp3)," AWS Documentation, 2024.
[19] Linux Kernel Org., "capabilities(7) — CAP_CHECKPOINT_RESTORE," 2024.